

QUEUE

Unit 6

Presented By: Saroj Bhandari

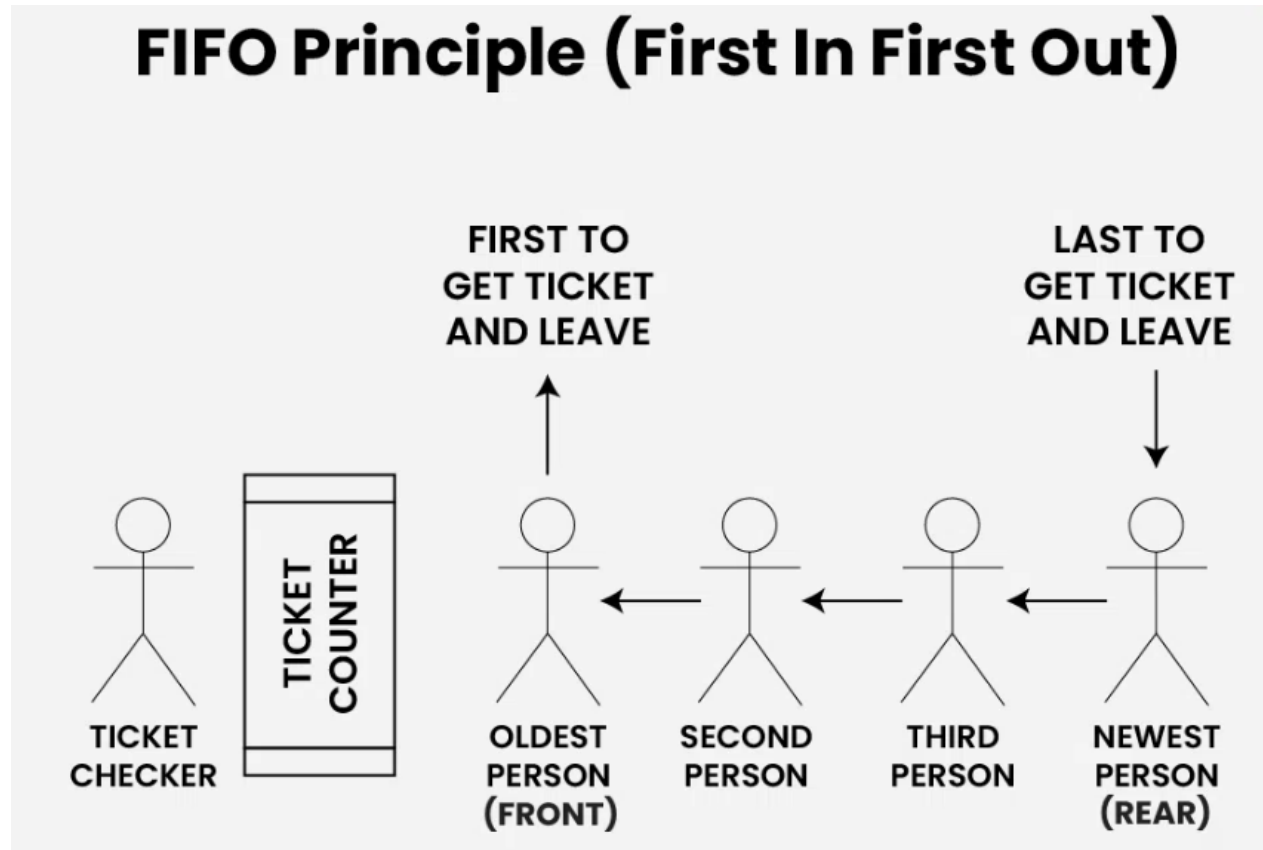
Syllabus

- Introduction
- Queue as an ADT
- Primitive Operations in Queue
- Linear and Circular Queue and Their Application
- Enqueue and Dequeue
- Priority Queue

Introduction to Queue Data Structure

- **Queue** is a linear data structure that follows **FIFO (First In First Out) Principle**, so the first element inserted is the first to be popped out.
- **FIFO Principle in Queue:**
 - FIFO Principle states that the first element added to the Queue will be the first one to be removed or processed. So, Queue is like a line of people waiting to purchase tickets, where the first person in line is the first person served. (i.e. First Come First Serve).

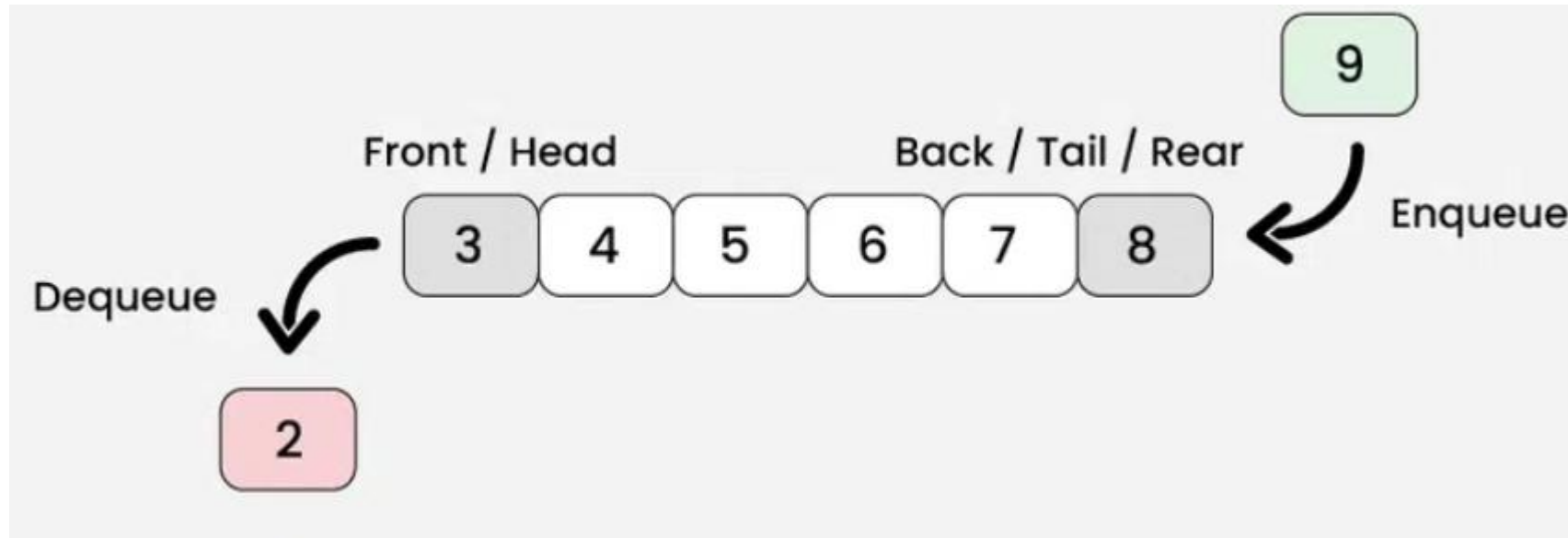
Example



Basic Terminologies of Queue

- **Front:** Position of the entry in a queue ready to be served, that is, the first entry that will be removed from the queue, is called the **front** of the queue. It is also referred as the **head** of the queue.
- **Rear:** Position of the last entry in the queue, that is, the one most recently added, is called the **rear** of the queue. It is also referred as the **tail** of the queue.
- **Size:** Size refers to the **current** number of elements in the queue.
- **Capacity:** Capacity refers to the **maximum** number of elements the queue can hold.

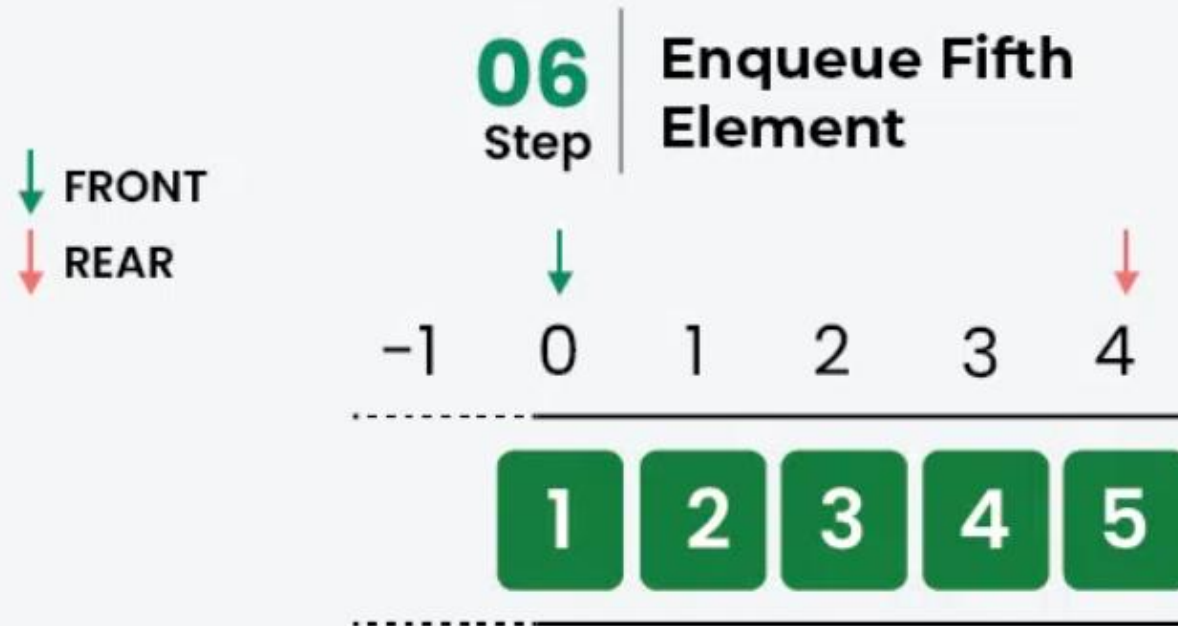
Representation of Queue



Operations on Queue

- **Enqueue:**
 - Enqueue operation **adds (or stores) an element to the end of the queue.**
- **Steps:**
 - Check if the **queue is full**. If so, return an **overflow** error and exit.
 - If the queue is **not full**, increment the **rear** pointer to the next available position.
 - Insert the element at the rear.

Enqueue

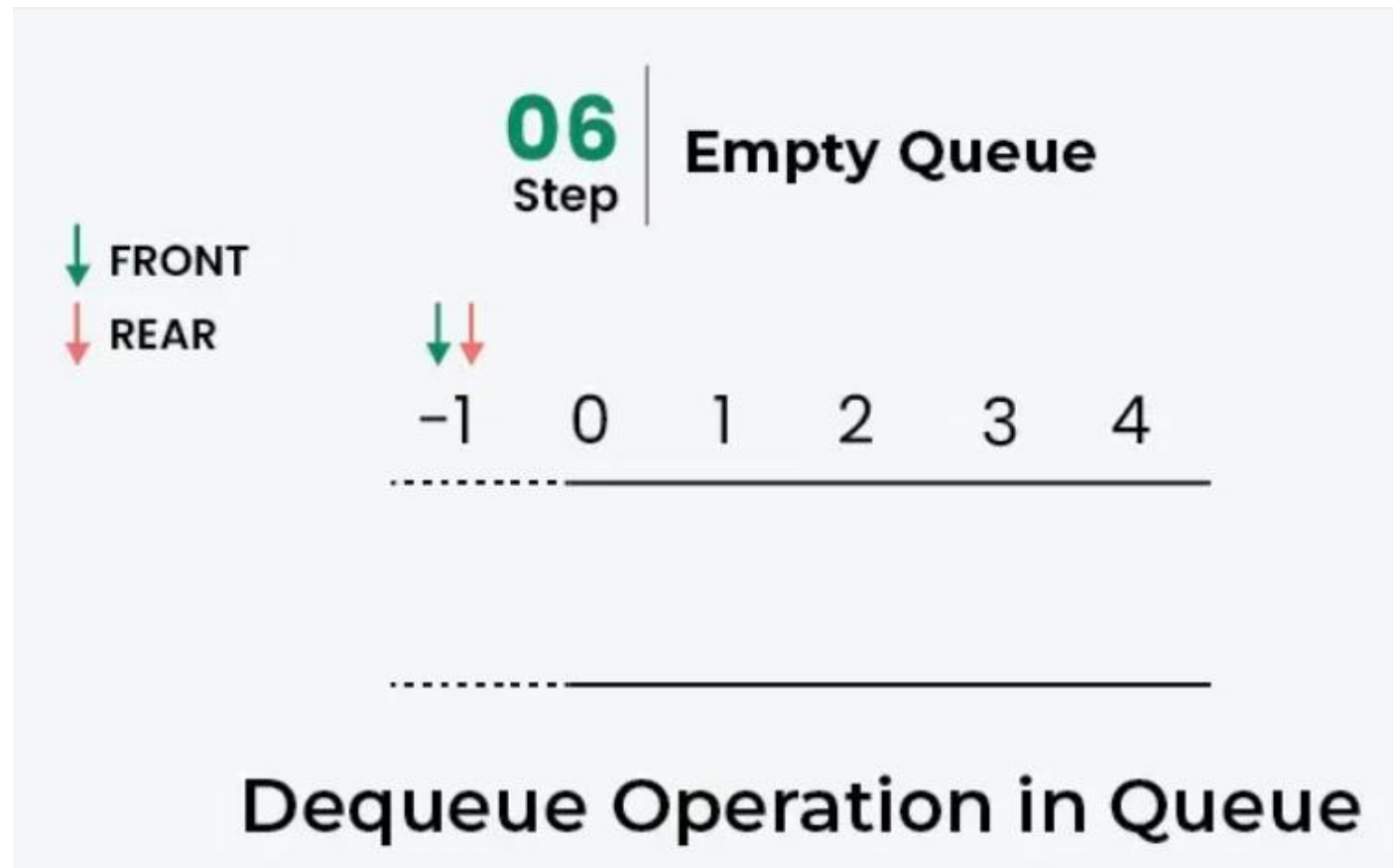


Enqueue Operation in Queue

2. Dequeue:

- Dequeue operation removes the element at the front of the queue. The following steps are taken to perform the dequeue operation:
 - Check if the **queue is empty**. If so, return an **underflow** error.
 - Remove the element at the **front**.
 - **Increment** the **front** pointer to the next element.

Example



Other Queue operations

- **peek() or front()-** Acquires the data element available at the front node of the queue without deleting it.
- **rear()** – This operation returns the element at the rear end without removing it.
- **isFull()** – Validates if the queue is full.
- **isEmpty()** – Checks if the queue is empty.
- **size():** This operation returns the size of the queue i.e. the total number of elements it contains.

The Queue Abstract Data Type

- An ADT defines the behavior of a data structure — what operations it supports — without specifying the implementation details.
 - **dequeue()** – Removal of elements from the queue.
 - **peek() or front()**- Acquires the data element available at the front node of the queue without deleting it.
 - **rear()** – This operation returns the element at the rear end without removing it.
 - **isFull()** – Validates if the queue is full.
 - **isEmpty()** – Checks if the queue is empty.
 - **size()**: This operation returns the size of the queue i.e. the total number of elements it contains.

Linear Queue

- **Recap**
 - A **Linear Queue** is a sequential collection that follows **FIFO (First In First Out)**. It works like a line — the first person in line is served first.
- **Key points to remember:**
 - **Front** points to the first element.
 - **Rear** points to the last inserted element.
 - **Enqueue** inserts elements at the rear.
 - **Dequeue** removes elements from the front.

Pros and cons of Linear Queue

Advantages:

- Simple and easy to implement.
- Maintains order (FIFO).

Disadvantages:

- Wastes space after multiple deletions (because the front moves forward but doesn't shift elements back).
- Once rear reaches the end, no new elements can be added — even if space is available at the front (solved by circular queues).

Algorithm for Linear Queue

Let's define the basic operations.

❑ Enqueue Operation (Insert)

1. Check if $\text{rear} == \text{size} - 1$. If true, Queue is Full.
2. If queue is not full, increase rear by 1.
3. Add the new element at $\text{queue}[\text{rear}]$.

❑ Dequeue Operation (Remove)

1. Check if $\text{front} == -1$ or $\text{front} > \text{rear}$. If true, Queue is Empty.
2. If not empty, increment front to remove the element.

❑ Display Operation

1. Loop from front to rear and print elements.

Lab2: Implementation of Linear Queue

Objective:

- To Understand the concept of Linear Queues and FIFO principle.
- To Implement Enqueue, Dequeue, and Display operations in C.

Application of Linear Queue



CPU Scheduling

Processes are executed in the order they arrive

Documents print in the order they're sent

Printer Queue



Call Center System

Calls are answered in the sequence they come in

Vehicles move in the order they arrive at traffic signals

Traffic Management



Task Scheduling in OS

Tasks are handled in the order they're added to the job queue

Ticketing systems serve customers in the order they request help

Customer Service Systems



Problems in Linear Queue

- In a **Linear Queue**, after multiple enqueue() and dequeue() operations, we face two main problems:

1. Wasted Space (Inefficient Memory Usage)

- When elements are dequeued from the front, those spaces become unusable — even though the queue may still have free space at the start.

```
Enqueue: [10, 20, 30, 40, 50]
```

```
Dequeue: [__, 20, 30, 40, 50] ← Empty spot can't be reused.
```

If the rear reaches the end ($\text{rear} == \text{size} - 1$), you can't enqueue anymore — even though there's space at the front.

Queue Reset Problem

- Once the rear reaches the end of the array, it **can't move further — even if space is available at the front.**

Lets take a queue with size 5.

1. Enqueue 5 elements:

```
Enqueue(10), Enqueue(20), Enqueue(30), Enqueue(40), Enqueue(50)
Queue: [10, 20, 30, 40, 50]
Front = 0, Rear = 4
```

3. Enqueue(60):

Queue says "Full" —
even though there's space at the front!

2 Dequeue Two Elments

```
Dequeue() → Removes 10
Dequeue() → Removes 20
Queue: [__, __, 30, 40, 50]
Front = 2, Rear = 4
```

This all happens because **Rear = size - 1** and can't move further.

Circular Queue

- A **Circular Queue** is an advanced version of the linear queue where the last position connects back to the first — forming a **circle**.
- This solves the **space wastage and Queue Reset** problem in linear queues.

Key Features

- **Rear wraps around** to the front when it reaches the end (hence circular).
- **Efficient space utilization** — no unused slots.
- **FIFO behavior** is maintained.

Circular Queue Algorithms

Enqueue Operation

1. Check if the queue is full: $(\text{rear} + 1) \% \text{size} == \text{front}$.
2. If not full:
 - Move rear to $(\text{rear} + 1) \% \text{size}$.
 - Insert the new element at `queue[rear]`.

Dequeue Operation

1. Check if the queue is empty: $\text{front} == -1$.
2. If not empty:
 - Remove `queue[front]`.
 - Move front to $(\text{front} + 1) \% \text{size}$.
 - If the queue becomes empty, reset front and rear to -1.

Lab 2: Circular Queue Implementation

- **To Understand** the concept of **Circular Queues** and how they overcome the limitations of linear queues.

Application of Circular Queue

Circular Queue Applications



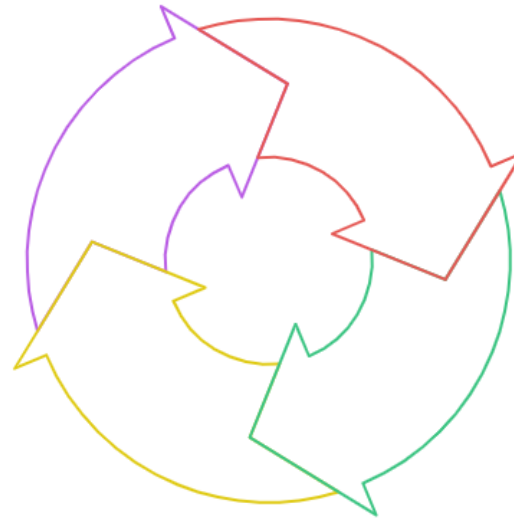
Music Playlist Queue

Loops playlists using circular queue logic.



Traffic Control Systems

Manages traffic flow with cyclic lights and queues.



CPU Task Scheduling

Ensures fair process execution using circular queues.



Memory Management

Utilizes circular queues for efficient buffering.

Priority Queue

- A **priority queue** is a type of queue that arranges elements based on their priority values.
- Each element has a priority associated. When we add an item, it is inserted in a position based on its priority.
- Elements with higher priority are typically retrieved or removed before elements with lower priority.
- **Binary heap** is the most common method to implement a priority queue. In binary heaps, we have easy access to the min (in min heap) or max (in max heap) and binary heap being a complete binary tree are easily implemented using arrays. Since we use arrays, we have cache friendliness advantage also.
- Priority Queue is used in algorithms such as Dijkstra's algorithm, Prim's algorithm, and Huffman Coding.

Types of Priority Queue

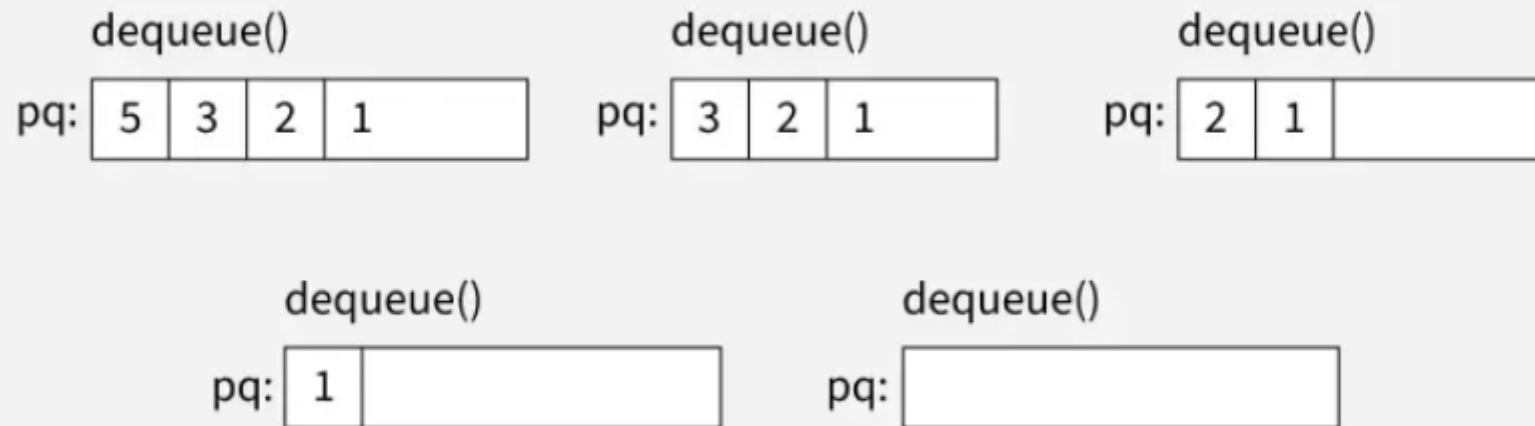


Ascending Order Priority Queue : In this queue, elements with lower values have higher priority. For example, with elements 4, 6, 8, 9, and 10, 4 will be dequeued first since it has the smallest value, and the dequeue operation will return 4.

Descending order Priority Queue : Elements with higher values have higher priority. The root of the heap is the highest element, and it is dequeued first. The queue adjusts by maintaining the heap property after each insertion or deletion.

Example

Implementation of Priority Queue



Priority Queue

Implementation of Priority Queue

1) Implement Priority Queue Using Array:

A simple implementation is to use an array of the following structure.

```
struct item {  
    int item;  
    int priority;  
}
```

- **enqueue()**: This function is used to insert new data into the queue.
- **dequeue()**: This function removes the element with the highest priority from the queue.
- **peek()/top()**: This function is used to get the highest priority element in the queue without removing it from the queue.

Linear Vs Circular Queue

Feature	Linear Queue	Circular Queue
Structure	Linear, with a fixed start and end	Circular, with the end connected back to the start
Representation	Array or linked list	Array or linked list
Memory Utilization	Inefficient, can lead to wastage of space	Efficient, utilizes all available space
Insertion (Enqueue)	Always at the rear	At the rear, but wraps around if the end of the array is reached and space is available at the front
Deletion (Dequeue)	Always from the front	From the front, but wraps around if necessary
Overflow Condition	Occurs when the rear reaches the maximum size of the array	Occurs when the queue is full and there is no space even after wrapping around
Underflow Condition	Occurs when trying to dequeue from an empty queue	Occurs when trying to dequeue from an empty queue
Front and Rear Indicators	Simple indices indicating the start and end positions	Indices that wrap around, requiring modulo operations
Implementation Complexity	Simpler	More complex due to the need to manage wrapping around
Example Use Cases	Simple scenarios where memory usage is not a concern	Scenarios where efficient memory usage is critical, such as in systems with limited memory

Multiple Queue

- ❑ **Multiple queues**, also known as multi-queues, refer to the concept of using multiple individual queues to manage different types of data or tasks, often with varying priorities or characteristics, within a single system.
- ❑ This approach allows for more efficient handling of diverse workloads by categorizing them and applying specific queue management strategies to each category.
- ❑ **Example:**
 - ❑ In a computing environment, different queues might be used for interactive processes (like user input), batch jobs (like background processing), or high-priority tasks.



Thank you !!!